

Adopt `not_fn` from Library Fundamentals 2 for C++17

Table of Contents

- [Introduction](#)
- [Recommended for Immediate Adoption](#)
- [Remove the Classic `not1` and `not2` negators](#)
- [Future Extensions](#)
- [Established Experience](#)
- [Proposed Wording](#)
- [Acknowledgements](#)
- [References](#)

Introduction

This paper recommends moving the `not_fn` function binder immediately out of Library Fundamentals 2 TS, into the C++17 Working paper. There are no parts of the Fundamentals TS that depend on this new component, and it serves a bigger purpose in the main standard, allowing the deprecation and removal of the last of the legacy function binder APIs.

Recommendation for Immediate Adoption

The function template `not_fn` is first proposed for Library Fundamentals 2, and so has not yet been published. However, there is extensive experience of this feature through the Boost library. The main reason cited for putting this into Fundamentals 2, rather than directly into C++17, was to give users early access to the feature. However, we are already seeing new C++17 library features available in the main standard library distributions, typically guarded by an experimental C++17 build mode flag.

The benefit of adopting this library directly into the C++ Standard is that it is the necessary missing component to allow us to deprecate the legacy negator types, `unary_negate` and `binary_negate`, and the factory functions `not1` and `not2`. These are the last two library components that depend on the adaptable function protocol of embedding typedefs in functor classes to support adaption. The remaining facilities such as `bind1st` have already been removed, following their deprecation in C++11, and adopting `not_fn` would allow us to reprecate, and ultimately remove, this final legacy.

Note that it is desirable to finally remove the legacy adaptable function protocol as it makes implementing several library types considerably more awkward with conditionally defined member typedefs, such as `std::function` wrapping a function type with exactly one or two parameters, or similarly for `std::reference_wrapper` for function references of exactly one or two arguments. Meanwhile, the adaptable protocol is not supported for newer language features, such as lambda expressions, and actually cannot be supported for some features such as polymorphic lambda expressions.

Remove the Classic `not1` and `not2` negators

Traditionally we would start a long deprecation process before removing supported library features such as `not1` and `not2`. However, this paper will argue that we should go further and actively remove them from the proposed C++17 standard, once the `not_fn` replacement is available, along with the adaptable functors have already been removed from the working paper.

The list of references at the end of this paper shows a number of other deprecated features targetted for removal in C++17, suggesting this would be an ideal time to perform cleanup, taking any hit of breaking compatibility with older code in one transition, rather than making each new version of the standard a risk.

One the last of the classic binders are removed, then all of the support machinery created only for them, that no longer functions as well as designed with the addition of lambda expressions and other recent language and library features, should also be removed. That means removing all of the `argument_type`, `first_argument_type`, and `second_argument_type` typedefs in the standard library, but *retaining* the `result_type` typedefs which are also part of the *INVOKE* protocol.

Future Extensions

Another feature of the Boost library that contributed the original design for `not_fn` is an expansion of the `bind` language to return a `bind` expression that negates its result when called as `!bind(a1, a2, ..., aN)`.

While this feature is desirable, it has not yet been through the LEWG process, and is slightly trickier to word than implement as the result of a 'bind' expression is unspecified, and the interaction works by providing an `operator!` overload for the unspecified `bind`-object type. Therefore, this is left as an extension to be proposed for the Library Fundamentals 3 TS.

Established Experience

The `not_fn` library, and its interaction with `bind`, have been popular parts of the Boost library distribution for over a decade now. The interface and specification are mature.

Proposed Wording

Amend the <function> header synopsis in **20.0p2 [function.objects]**:

```
// 20.9.9, negators:
template <class Predicate> class unary_negate;
template <class Predicate>
    constexpr unary_negate<Predicate> not1(const Predicate&);
template <class Predicate> class binary_negate;
template <class Predicate>
    constexpr binary_negate<Predicate> not2(const Predicate&);

// 20.9.9, Function template not_fn
template <class F> unspecified not_fn(F&& f);
```

Strike the last note in 20.9p5 [function.objects]:

5 [Note: To enable adaptors and other components to manipulate function objects that take one or two arguments many of the function objects in this clause correspondingly provide typedefs `argument_type` and `result_type` for function objects that take one argument and `first_argument_type`, `second_argument_type`, and `result_type` for function objects that take two arguments. *end note*]

Simplify the definition of `reference_wrapper`:

20.9.4 Class template `reference_wrapper` [refwrap]

```
namespace std {
    template <class T> class reference_wrapper {
    public :
        // types
        typedef T type;
        typedef see below result_type;           // not always defined
        typedef see below argument_type;         // not always defined
        typedef see below first_argument_type;   // not always defined
        typedef see below second_argument_type;  // not always defined

        // construct/copy/destroy
        reference_wrapper(T&) noexcept;
        reference_wrapper(T&&) = delete; // do not bind to temporary objects
        reference_wrapper(const reference_wrapper& x) noexcept;

        // assignment
        reference_wrapper& operator=(const reference_wrapper& x) noexcept;

        // access
        operator T& () const noexcept;
        T& get() const noexcept;

        // invocation
        template <class... ArgTypes>
        result_of_t<T&(ArgTypes&&...)>
        operator() (ArgTypes&&...) const;
    };
}
```

1 `reference_wrapper<T>` is a `CopyConstructible` and `CopyAssignable` wrapper around a reference to an object or function of type `T`.

2 `reference_wrapper<T>` shall be a trivially copyable type (3.9).

3 reference_wrapper<T> has a weak result type (20.9.2). If T is a function type, result_type shall be a synonym for the return type of T.

4 The template specialization reference_wrapper<T> shall define a nested type named argument_type as a synonym for T1 only if the type T is any of the following:

(4.1) "a function type or a pointer to function type taking one argument of type T1

(4.2) "a pointer to member function R T0::f cv (where cv represents the member function's cv-qualifiers); the type T1 is cv T0*

(4.3) "a class type where the qualified-id T::argument_type is valid and denotes a type (14.8.2); the type T1 is T::argument_type;

5 The template instantiation reference_wrapper<T> shall define two nested types named first_argument_type and second_argument_type as synonyms for T1 and T2, respectively, only if the type T is any of the following:

(5.1) "a function type or a pointer to function type taking two arguments of types T1 and T2

(5.2) "a pointer to member function R T0::f(T2) cv (where cv represents the member function's cv-qualifiers); the type T1 is cv T0*

(5.3) "a class type where the qualified-ids T::first_argument_type and T::second_argument_type are both valid and both denote types (14.8.2); the type T1 is T::first_argument_type and the type T2 is T::second_argument_type;

Replace subclause 20.9.9 [negators] with a new subclause 20.9.9 [func.notfn], copying everything from the Library Fundamentals 2 TS other than the struck-out note:

20.9.9 Negators [negators]

1 Negators not1 and not2 take a unary and a binary predicate, respectively, and return their complements (5.3.1).

```
template <class Predicate>
class unary_negate {
public:
    constexpr explicit unary_negate(const Predicate& pred);
    constexpr bool operator()(const typename Predicate::argument_type& x) const;
    typedef typename Predicate::argument_type argument_type;
    typedef bool result_type;
};
```

2 operator() returns !pred(x);

```
template <class Predicate>
    constexpr unary_negate<Predicate> not1(const Predicate& pred);
```

3 Returns: unary_negate<Predicate>(pred);

```
template <class Predicate>
class binary_negate {
public:
    constexpr explicit binary_negate(const Predicate& pred);
    constexpr bool operator()(const typename Predicate::first_argument_type& x,
        const typename Predicate::second_argument_type& y) const;
    typedef typename Predicate::first_argument_type first_argument_type;
    typedef typename Predicate::second_argument_type second_argument_type;
    typedef bool result_type;
};
```

4 operator() returns !pred(x,y);

```
template <class Predicate>
    constexpr binary_negate<Predicate> not2(const Predicate& pred);
```

5 Returns: binary_negate<Predicate>(pred);

20.9.9 Function template not fn [func.not fn]

```
template <class F> unspecified not_fn(F&& f);
```

1 In the text that follows:

- FD is the type decay `t<F>`,
- fd is an lvalue of type FD constructed from `std::forward<F>(f)`,
- fn is a forwarding call wrapper created as a result of `not_fn(f)`,

2 Requires: `is_constructible<FD, F>::value` shall be true. fd shall be a callable object (Â§20.9.1).

3 Returns: A forwarding call wrapper fn such that the expression `fn(a1, a2, ..., aN)` is equivalent to `!INVOKE(fd, a1, a2, ..., aN)` (Â§20.9.2).

4 Throws: Nothing unless the construction of fd throws an exception.

5 Remarks: The return type shall satisfy the requirements of `MoveConstructible`. If FD satisfies the requirements of `CopyConstructible`, then the return type shall satisfy the requirements of `CopyConstructible`. [Note: This implies that FD is `MoveConstructible`. â€” end note]

~~[Note: Function template `not_fn` can usually provide a better solution than using the negators `not1` and `not2` â€” end note]~~

Strike the unnecessary typedefs from the unspecified return-type of `mem_fn` (note that the first bullet continues to provide `result_type` when the second and third bullets are removed):

20.9.11 Function template `mem_fn` [func.memfn]

```
template<class R, class T> unspecified mem_fn(R T::* pm);
```

1 Returns: A simple call wrapper (20.9.1) `fn` such that the expression `fn(t, a2, ..., aN)` is equivalent to `INVOKE (pm, t, a2, ..., aN)` (20.9.2). `fn` shall have a nested type `result_type` that is a synonym for the return type of `pm` when `pm` is a pointer to member function.

~~2 The simple call wrapper shall define two nested types named `argument_type` and `result_type` as synonyms for `cv T*` and `Ret`, respectively, when `pm` is a pointer to member function with `cv`-qualifier `cv` and taking no arguments, where `Ret` is `pm`'s return type.~~

~~3 The simple call wrapper shall define three nested types named `first_argument_type`, `second_argument_type`, and `result_type` as synonyms for `cv T*`, `T1`, and `Ret`, respectively, when `pm` is a pointer to member function with `cv`-qualifier `cv` and taking one argument of type `T1`, where `Ret` is `pm`'s return type.~~

4 Throws: Nothing.

Strike the typedefs from the class definition of `function`:

20.9.12.2 Class template function [func.wrap.func]

```
namespace std {
template<class> class function; // undefined
template<class R, class... ArgTypes>
class function<R(ArgTypes...)> {
public:
    typedef R result_type;
    typedef T1 argument_type; // only if sizeof...(ArgTypes) == 1 and
                             // the type in ArgTypes is T1
    typedef T1 first_argument_type; // only if sizeof...(ArgTypes) == 2 and
                                   // ArgTypes contains T1 and T2
    typedef T2 second_argument_type; // only if sizeof...(ArgTypes) == 2 and
                                    // ArgTypes contains T1 and T2

    // further details elided...
};
```

Amend bullet (1.3), on the definition of standard hash functors:

20.9.13 Class template hash [unord.hash]

1 The unordered associative containers defined in 23.5 use specializations of the class template `hash` as the default hash function. For all object types `Key` for which there exists a specialization `hash<Key>`, and for all enumeration types

(7.2) Key, the instantiation hash<Key> shall:

(1.1) “satisfy the Hash requirements (17.6.3.4), with Key as the function call argument type, the DefaultConstructible requirements (Table 19), the CopyAssignable requirements (Table 23),

(1.2) “be swappable (17.6.3.2) for lvalues,

~~(1.3) “provide two nested types result_type and argument_type which shall be synonyms for size_t and Key, respectively,~~

(1.3) “provide a nested type, result_type, which shall be synonyms for size_t,

(1.4) “satisfy the requirement that if $k_1 == k_2$ is true, $h(k_1) == h(k_2)$ is also true, where h is an object of type hash and k_1 and k_2 are objects of type Key;

(1.5) “satisfy the requirement that the expression $h(k)$, where h is an object of type hash and k is an object of type Key, shall not throw an exception unless hash is a user-defined specialization that depends on at least one user-defined type.

Strike the argument_type, first_argument_type, and second_argument_type typedefs from all standard library classes, including (but not limited to):

- 20.8.2.4 owner_lessx
- 20.9.5 plus
- 20.9.5 minus
- 20.9.5 multiplies
- 20.9.5 divides
- 20.9.5 modulus
- 20.9.5 negate
- 20.9.6 equal_to
- 20.9.6 not_equal_to
- 20.9.6 greater
- 20.9.6 less
- 20.9.6 greater_equal
- 20.9.6 less_equal
- 20.9.7 logical_and
- 20.9.7 logical_or
- 20.9.7 logical_not
- 20.9.8 bit_and
- 20.9.8 bit_or
- 20.9.8 bit_xor
- 20.9.8 bit_not
- 23.4.4.1 map::compare
- 23.4.5.1 multimap::compare

Optionally excise class 3 from the Library Fundamentals 2 TS, as it is no longer required (section numbering per [n4481](#)).

Acknowledgements

Thanks to Tomasz Kamiński for writing up and proposing this feature for the Library Fundamentals TS, this is essentially his work that I am trying to promote!

References

- [N4076](#) A proposal to add a generalized callable negator, Tomasz Kamiński
- [N4086](#) Removing trigraphs?!, Richard Smith
- [N4168](#) Removing auto_ptr, Billy Baker
- [N4190](#) Removing auto_ptr, random_shuffle(), And Old <functional> Stuff, Stephan T. Lavavej
- [P0001R0](#) Remove Deprecated Use of the register Keyword
- [P0002R0](#) Remove Deprecated operator++(bool)
- [P0003R0](#) Remove Deprecated Exception Specifications from C++17
- [P0004R0](#) Remove Deprecated iostreams aliases